ATNLP: Lecture 19
Syntax and parsing

Shay Cohen

March 2, 2026

School of informatics

History?

- (circa 2011) Computing some sort of meaning?
- What did it require?

Syntax

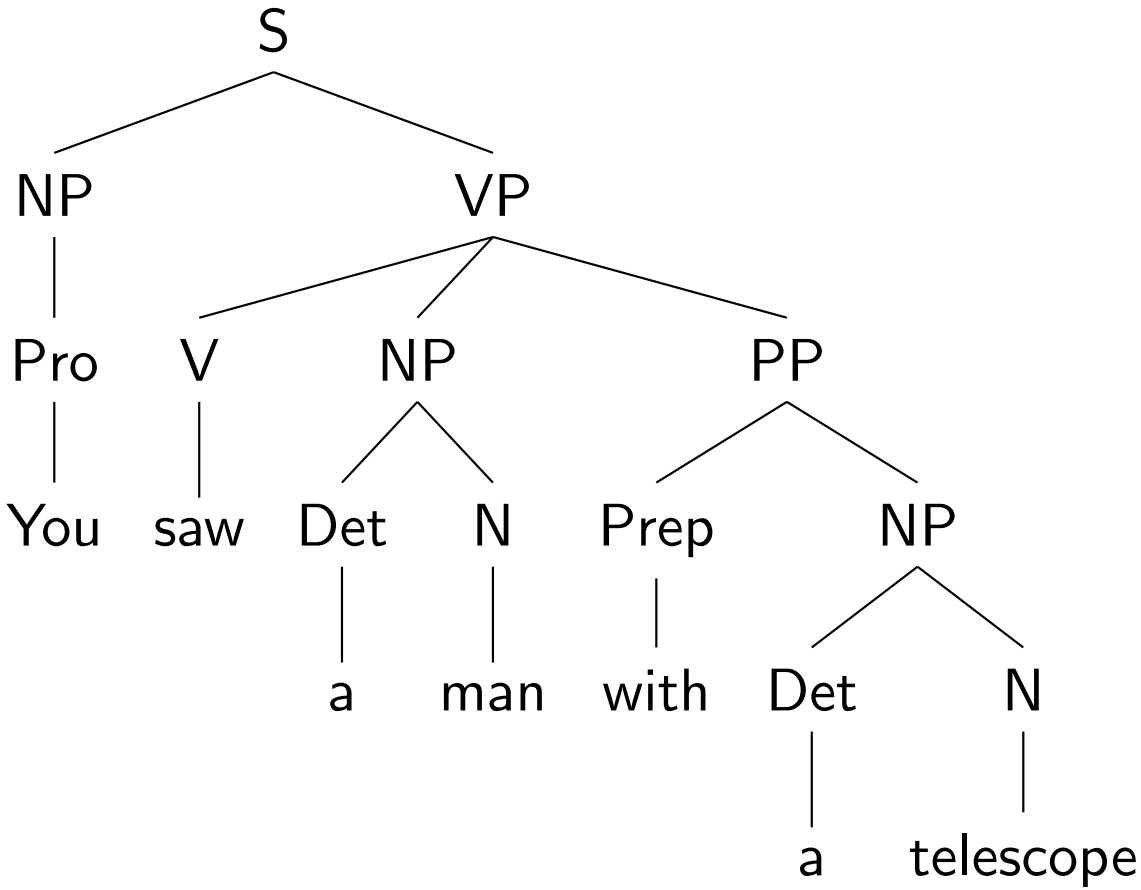
- Syntactic understanding of text was needed as a scaffold for understanding meaning
- Understanding the syntactic structure of text, in a sense, "shortcuts" distances between words and phrases in (otherwise sequential) text, providing direct relationship between them that can be further processed
- LLMs taught us that much of this shortcutting can be done implicitly (in one form or the other), without requiring explicit processing

Motivation

- While syntax is not explicitly used now in LLMs, it is an excellent example for structured prediction, and many algorithms in the area of structured prediction are inspired by parsing
- Low-resource multilingual settings
- Constrained decoding when decoding format is known (text normalisation?)
- Synthetic data
- Understanding how similar LLM language processing is to human processing (?)

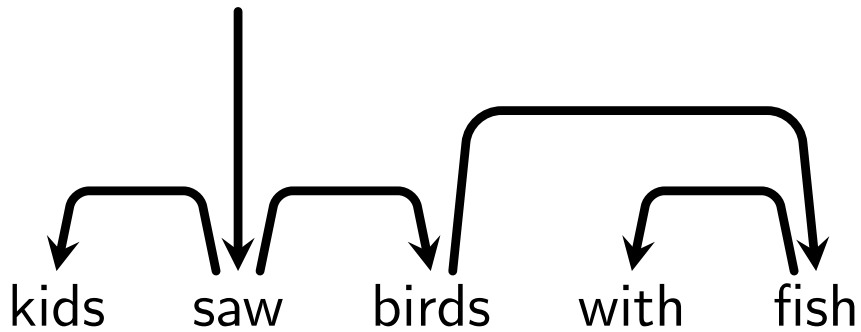
To ground things

Phrase-structure trees:



To ground things [2]

Dependency trees:

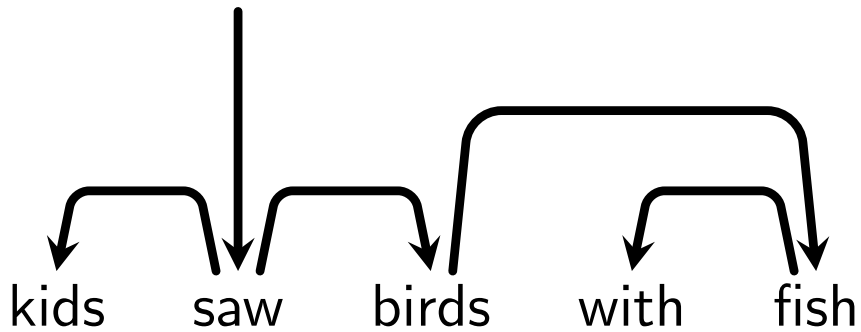


Edges can also be labelled with types of relations (subj, obj)

Food for thought: For what languages would dependency tree modelling be more suitable?

To ground things [2]

Dependency trees:



Edges can also be labelled with types of relations (subj, obj)

Food for thought: For what languages would dependency tree modelling be more suitable? Free word order languages

Desirable Properties of a Grammar

Chomsky specified two properties that make a grammar "interesting and satisfying":

- It should be a finite specification of the strings of the language, rather than a list of its sentences.
- It should be revealing, in allowing strings to be associated with meaning (semantics) in a systematic way.

We can add another desirable property:

- It should capture structural and distributional properties of the language. (E.g. where heads of phrases are located; how a sentence transforms into a question; which phrases can float around the sentence.)

Example grammar

$S \rightarrow NP\ VP$	[.80]	$Det \rightarrow the$	[.10]
$S \rightarrow Aux\ NP\ VP$	[.15]	$Det \rightarrow a$	[.90]
$S \rightarrow VP$	[.05]	$Noun \rightarrow book$	[.10]
$NP \rightarrow Pronoun$	[.35]	$Noun \rightarrow flight$	[.30]
$NP \rightarrow Proper\text{-}Noun$	[.30]	$Noun \rightarrow dinner$	[.60]
$NP \rightarrow Det\ Nominal$	[.20]	$Proper\text{-}Noun \rightarrow Houston$	[.60]
$NP \rightarrow Nominal$	[.15]	$Proper\text{-}Noun \rightarrow NWA$	[.40]
$Nominal \rightarrow Noun$	[.75]	$Aux \rightarrow does$	[.60]
$Nominal \rightarrow Nominal\ Noun$	[.05]	$Aux \rightarrow can$	[.40]
$VP \rightarrow Verb$	[.35]	$Verb \rightarrow book$	[.30]
$VP \rightarrow Verb\ NP$	[.20]	$Verb \rightarrow include$	[.30]
$VP \rightarrow Verb\ NP\ PP$	[.10]	$Verb \rightarrow prefer$	[.20]
$VP \rightarrow Verb\ PP$	[.15]	$Verb \rightarrow sleep$	[.20]

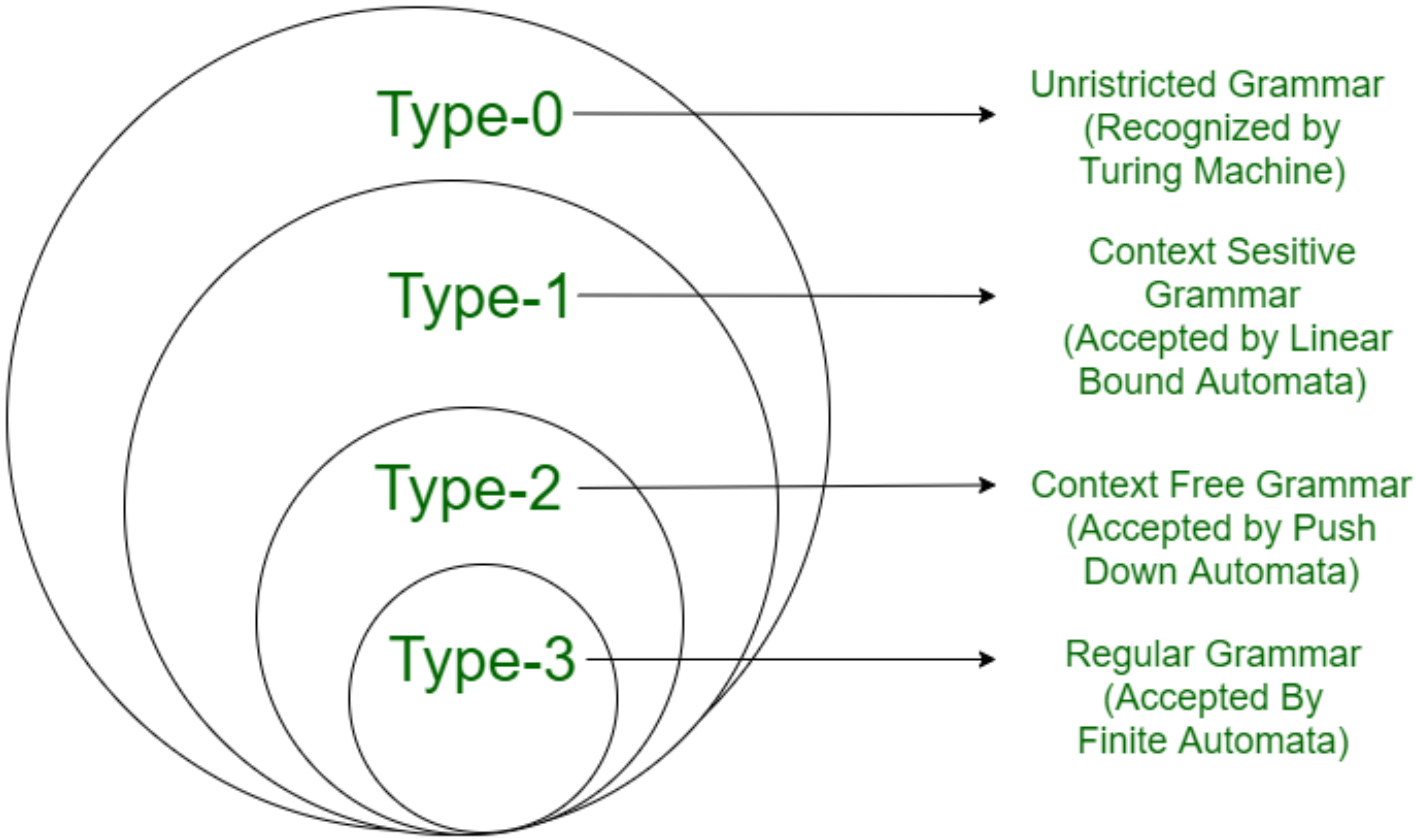
Food for thought: Why does it satisfy the two desiderata?

Example grammar

$S \rightarrow NP\ VP$	[.80]	$Det \rightarrow the$	[.10]
$S \rightarrow Aux\ NP\ VP$	[.15]	$Det \rightarrow a$	[.90]
$S \rightarrow VP$	[.05]	$Noun \rightarrow book$	[.10]
$NP \rightarrow Pronoun$	[.35]	$Noun \rightarrow flight$	[.30]
$NP \rightarrow Proper\text{-}Noun$	[.30]	$Noun \rightarrow dinner$	[.60]
$NP \rightarrow Det\ Nominal$	[.20]	$Proper\text{-}Noun \rightarrow Houston$	[.60]
$NP \rightarrow Nominal$	[.15]	$Proper\text{-}Noun \rightarrow NWA$	[.40]
$Nominal \rightarrow Noun$	[.75]	$Aux \rightarrow does$	[.60]
$Nominal \rightarrow Nominal\ Noun$	[.05]	$Aux \rightarrow can$	[.40]
$VP \rightarrow Verb$	[.35]	$Verb \rightarrow book$	[.30]
$VP \rightarrow Verb\ NP$	[.20]	$Verb \rightarrow include$	[.30]
$VP \rightarrow Verb\ NP\ PP$	[.10]	$Verb \rightarrow prefer$	[.20]
$VP \rightarrow Verb\ PP$	[.15]	$Verb \rightarrow sleep$	[.20]

Food for thought: Why does it satisfy the two desiderata? Recursiveness (yet finite grammar), interpretation of the categories

Side note: Chomsky hierarchy



Parser learning

Parser: given a sentence, output its corresponding syntactic tree (phrase structure)
Two main schemes:

- Unsupervised parsing - learning only from text to create a parser

- Supervised parsing - learning from annotated data (such as a treebank) to create a parser

We will focus on the latter. While there is still work about doing neural unsupervised parsing, this problem is less in the focus in the current cycle of NLP. In English, most work uses the Penn Treebank (Marcus et al., 1993)

Early "neural" parsing

"Grammar as a foreign language" (Vinyals et al., 2015):

- Simply use a seq2seq model (LSTM) to take as input a sentence and return its corresponding tree
- Learn based on a treebank (Penn Treebank, for example)
- **Food for thought:** But a tree is a nested structure. And seq2seq returns sequences. What do we do?

Early "neural" parsing

"Grammar as a foreign language" (Vinyals et al., 2015):

- Simply use a seq2seq model (LSTM) to take as input a sentence and return its corresponding tree
- Learn based on a treebank (Penn Treebank, for example)
- **Food for thought:** But a tree is a nested structure. And seq2seq returns sequences. What do we do? Linearisation of the tree:
  (S (NP (Pro You)) (VP (V saw) (NP (Det a) (N man) (PP (P with) (Det a) (N telescope)))))
  We can simplify it by stripping out the words:
  (S (NP Pro) (VP V (NP Det N (PP P Det N))))
  And we can make it easier to process by annotating also the closing brackets:
  (S (NP Pro)$_{\text{NP}}$ (VP V (NP Det N (PP P Det N)$_{\text{PP}}$)$_{\text{NP}}$)$_{\text{VP}}$)$_{\text{S}}$

Results

That's really all there is to this parser, almost. No grammar!

Food for thought: Any issue with linearisation?

Results

That's really all there is to this parser, almost. No grammar!

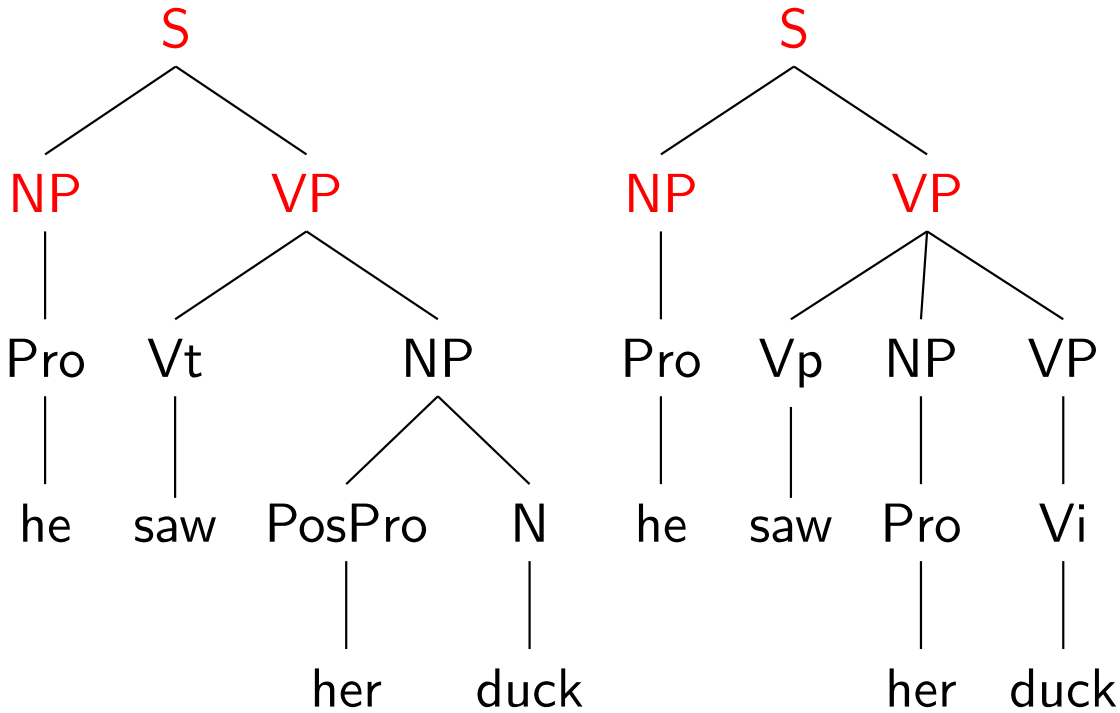
Food for thought: Any issue with linearisation? Nested brackets. Relatively rare to have "wrong" brackets, and if it happens, can add closing brackets or opening brackets as a "posthoc" solution.

Results:

Parser	Training Set	WSJ 22	WSJ 23
baseline LSTM+D	WSJ only	< 70	< 70
LSTM+A+D	WSJ only	88.7	88.3
LSTM+A+D ensemble	WSJ only	90.7	90.5
baseline LSTM	BerkeleyParser corpus	91.0	90.5
LSTM+A	high-confidence corpus	**92.8**	**92.1**
Petrov et al. (2006) [12]	WSJ only	91.1	90.4
Zhu et al. (2013) [13]	WSJ only	N/A	90.4
Petrov et al. (2010) ensemble [14]	WSJ only	92.5	91.8
Zhu et al. (2013) [13]	semi-supervised	N/A	91.3
Huang & Harper (2009) [15]	semi-supervised	N/A	91.3
McClosky et al. (2006) [16]	semi-supervised	92.4	**92.1**

Evaluating parse accuracy

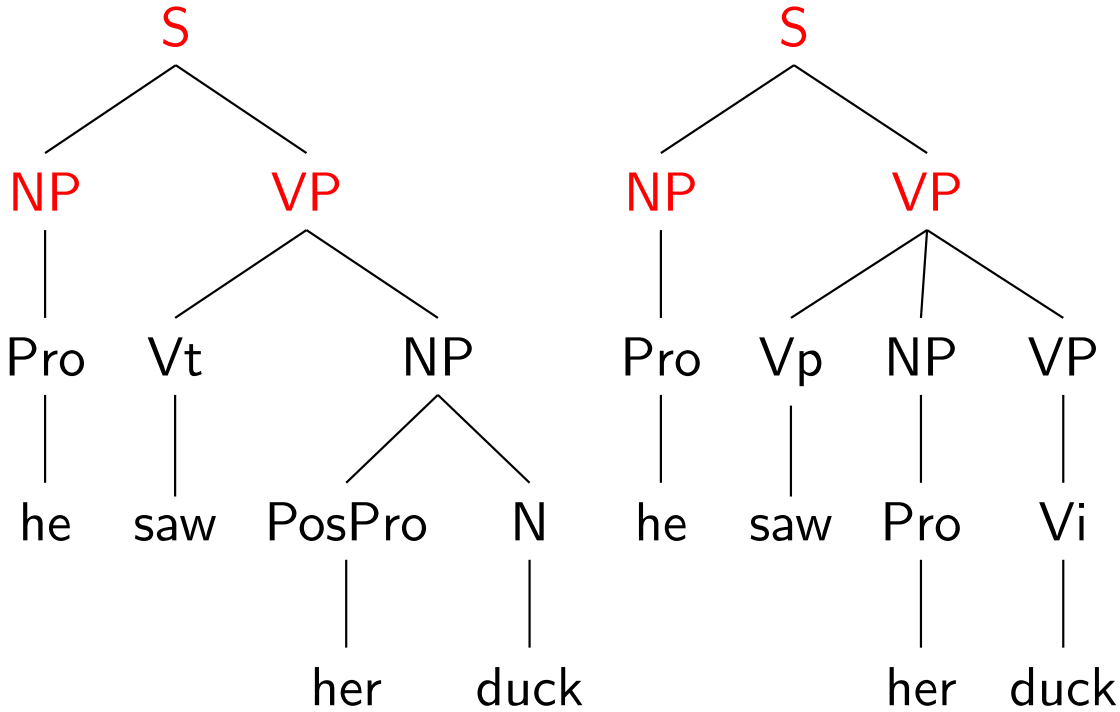
Compare **gold standard** tree (left) to **parser output** (right):



- Output constituent is counted correct if there is a gold constituent that spans the same sentence positions.
- Harsher measure: also require the constituent labels to match.
- Pre-terminals don't count as constituents.

Evaluating parse accuracy

Compare **gold standard** tree (left) to **parser output** (right):



- **Precision**: (# correct constituents)/(# in parser output) = 3/5
- **Recall**: (# correct constituents)/(# in gold standard) = 3/4
- **F-score**: balances precision/recall: 2pr/(p+r)

Parsing with Transformers

Could we use transformers for parsing, wouldn't that work even better? Yes! We will briefly look at Kitaev and Klein (2018):

- They use a transformer to encode the input.
- This results in a "context aware summary vector" (embedding) for each input word.
- The embedding encodes word, PoS tag, and position information.
- The embedding layers are combined to obtain span scores.
- The attention blocks and the attention heads are just like in Vaswani et al. (2017)
- But they also try factored attention heads, which separate position and content information.

Parsing with Transformers

The decoder of Kitaev and Klein (2018) works as follows:
A real-valued score $s(T)$ is assigned to a tree T:

$$s(T) = \sum_{(i,j,\ell) \in T} s(i,j,\ell)$$

where $s(i,j,\ell)$ is a score for the constituent located between position i and position j with label ℓ.

At test time, we compute:

$$\hat{T} = \operatorname*{argmax}_{T} s(T)$$

This can be found efficiently using CYK. This gives us the optimal output sequence, unlike beam search!

An important tangent

Dynamic programming / specifically CYK:
We want to compute:

$$\hat{T} = \operatorname*{argmax}_{T} s(T)$$

where

$$s(T) = \sum_{(i,j,\ell) \in T} s(i,j,\ell)$$

Class exercise: How do we solve this? Input: for each pairs of indices in the sentence we are given a score with a label, and we want to find the "proper" tree that has the highest total sum of scores.

Dynamic programming

Break a problem into sub-problems of the same type, solve them (by breaking them too to even smaller sub-problems, ...) and then combine their solution into a solution for the current level

In the case of parsing, we hold a a data structure called a chart $A(i, j, \ell)$ which will hold the score of the "best" tree that spans indices i through j with label l at the top.

$$A(i, j, \ell) = \max_{k, \ell_1, \ell_2} A(i, k, \ell_1) + A(k, j, \ell_2) + s(i, j, \ell)$$

k ranges over mid-points between i and j
The idea: The best tree for the current level is going to be made from *some* two other best sub-trees for *some* midpoint
Food for thought: Why is this the case?

Dynamic programming

Break a problem into sub-problems of the same type, solve them (by breaking them too to even smaller sub-problems, ...) and then combine their solution into a solution for the current level

In the case of parsing, we hold a a data structure called a chart $A(i, j, \ell)$ which will hold the score of the "best" tree that spans indices i through j with label l at the top.

$$A(i, j, \ell) = \max_{k, \ell_1, \ell_2} A(i, k, \ell_1) + A(k, j, \ell_2) + s(i, j, \ell)$$

k ranges over mid-points between i and j

The idea: The best tree for the current level is going to be made from *some* two other best sub-trees for *some* midpoint

Food for thought: Why is this the case? For a given tree, if we were not using the best sub-trees, we could make that tree achieve a higher score

Food for thought: How do we get a tree from that?

Dynamic programming

Break a problem into sub-problems of the same type, solve them (by breaking them too to even smaller sub-problems, ...) and then combine their solution into a solution for the current level

In the case of parsing, we hold a a data structure called a chart $A(i, j, \ell)$ which will hold the score of the "best" tree that spans indices i through j with label l at the top.

$$A(i, j, \ell) = \max_{k, \ell_1, \ell_2} A(i, k, \ell_1) + A(k, j, \ell_2) + s(i, j, \ell)$$

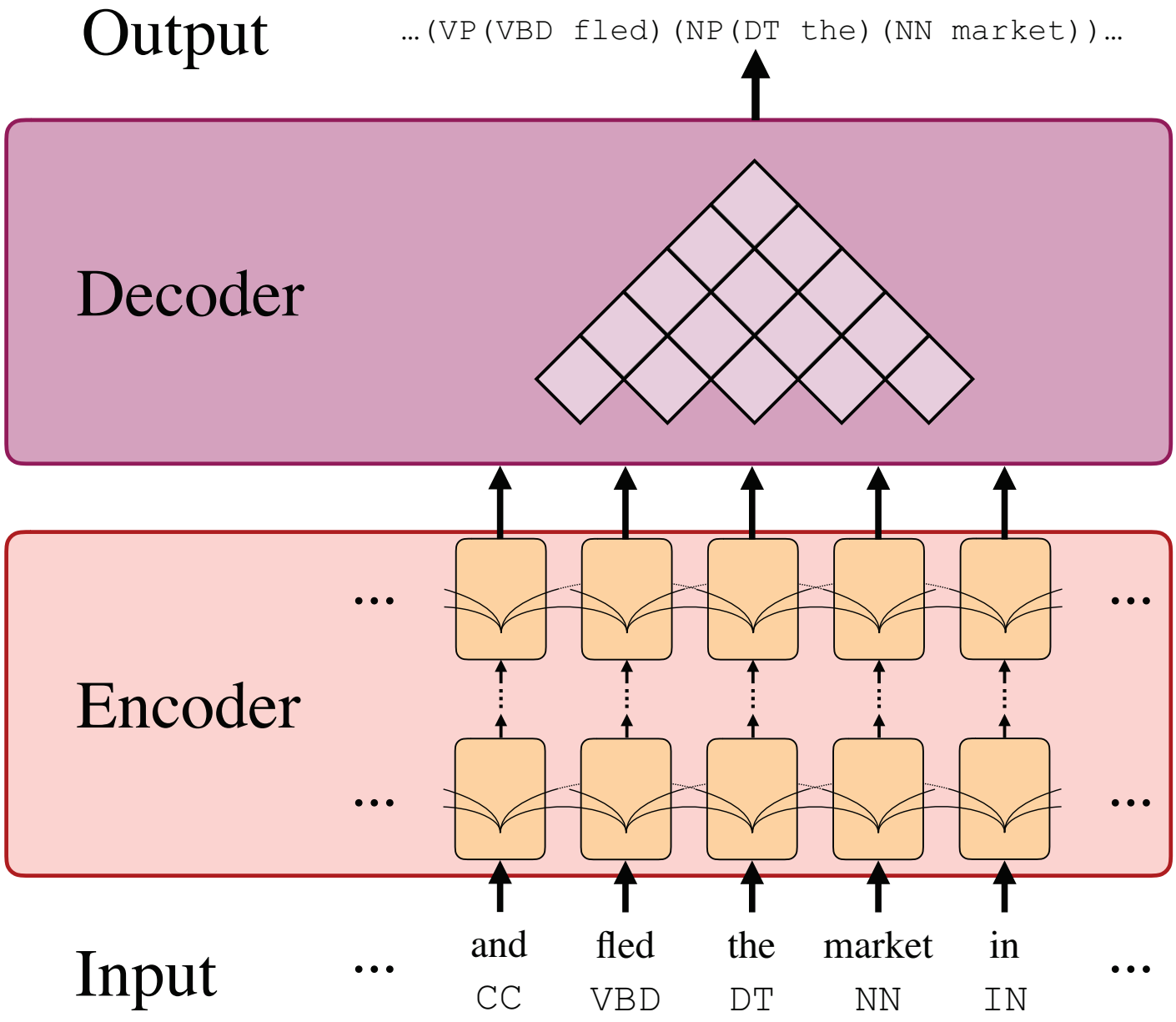
k ranges over mid-points between i and j

The idea: The best tree for the current level is going to be made from *some* two other best sub-trees for *some* midpoint

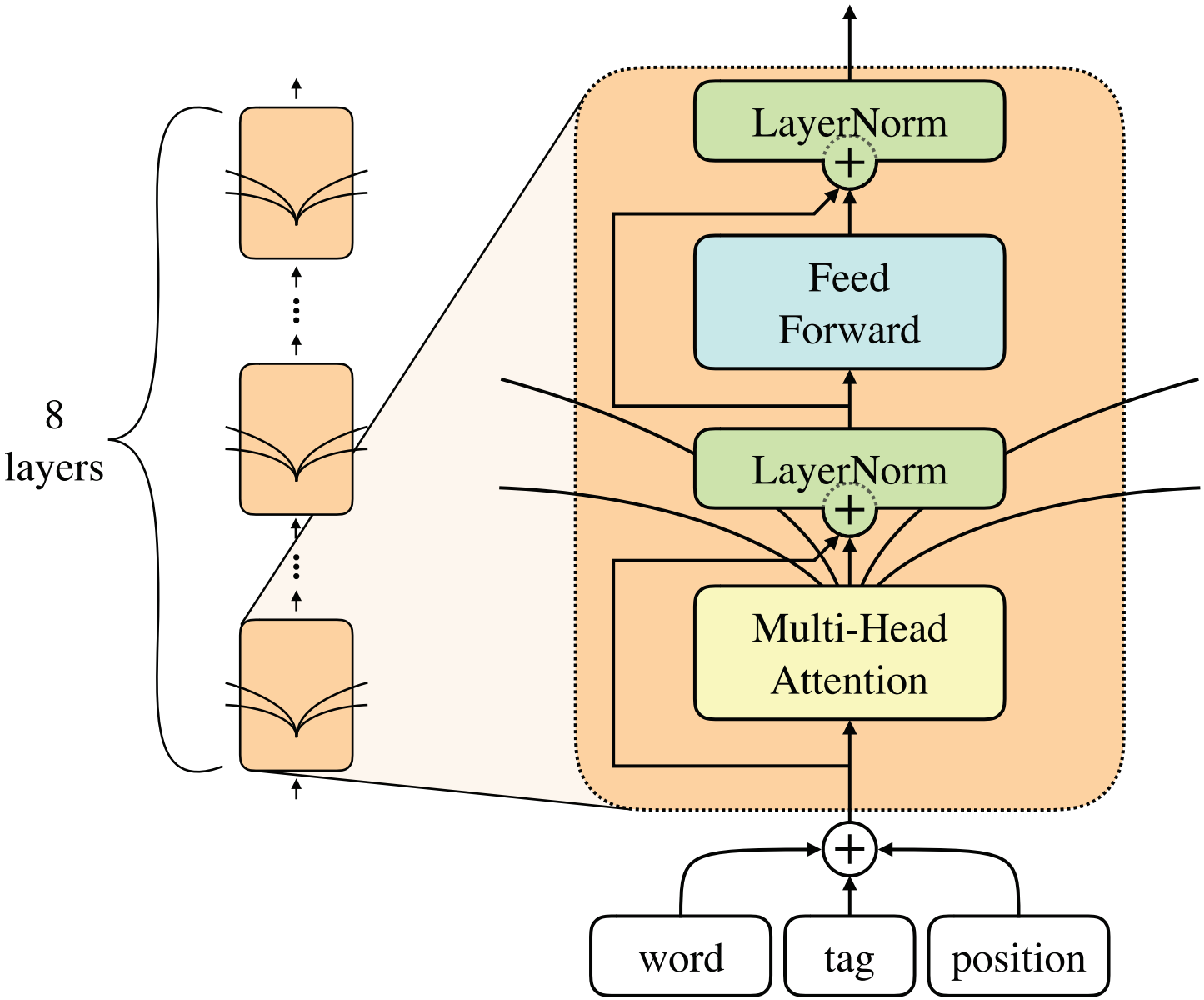
Food for thought: Why is this the case? For a given tree, if we were not using the best sub-trees, we could make that tree achieve a higher score

Food for thought: How do we get a tree from that? Backpointers

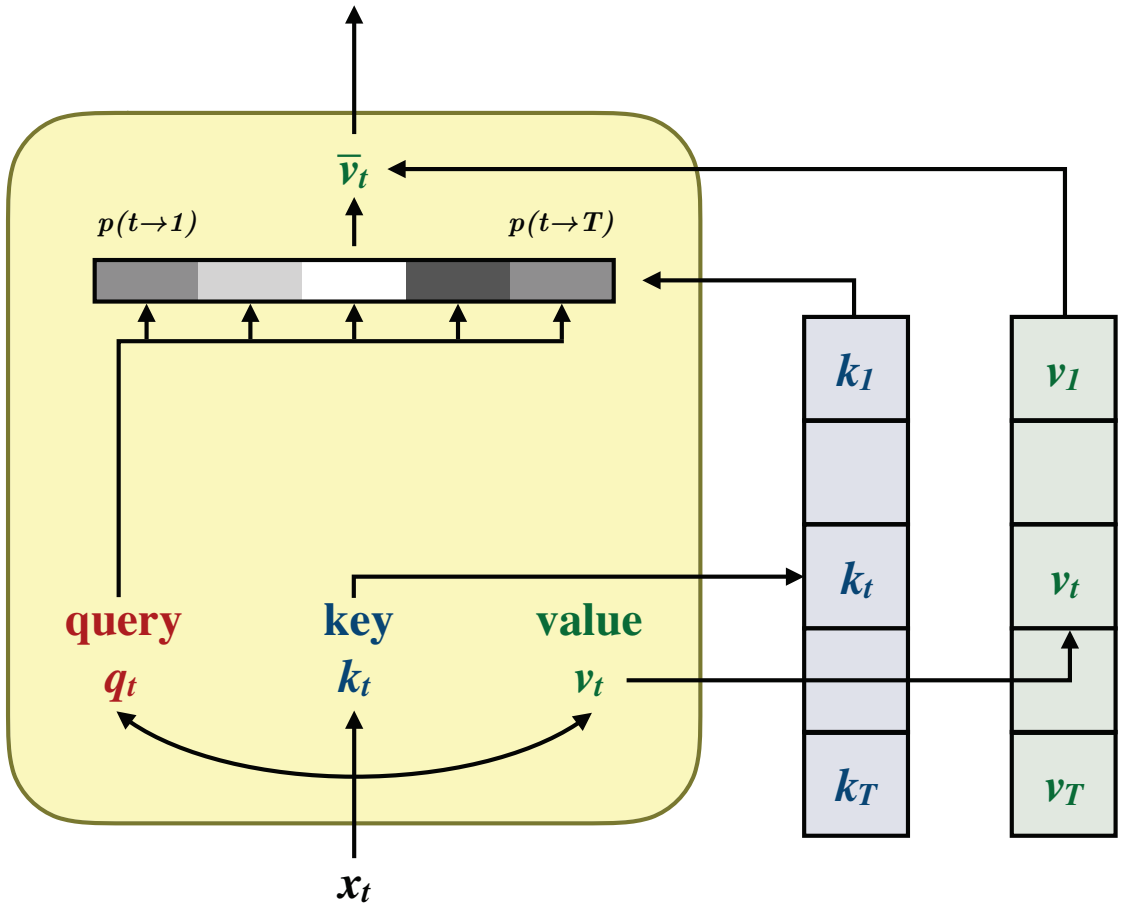
Overall Architecture



Transformer Block

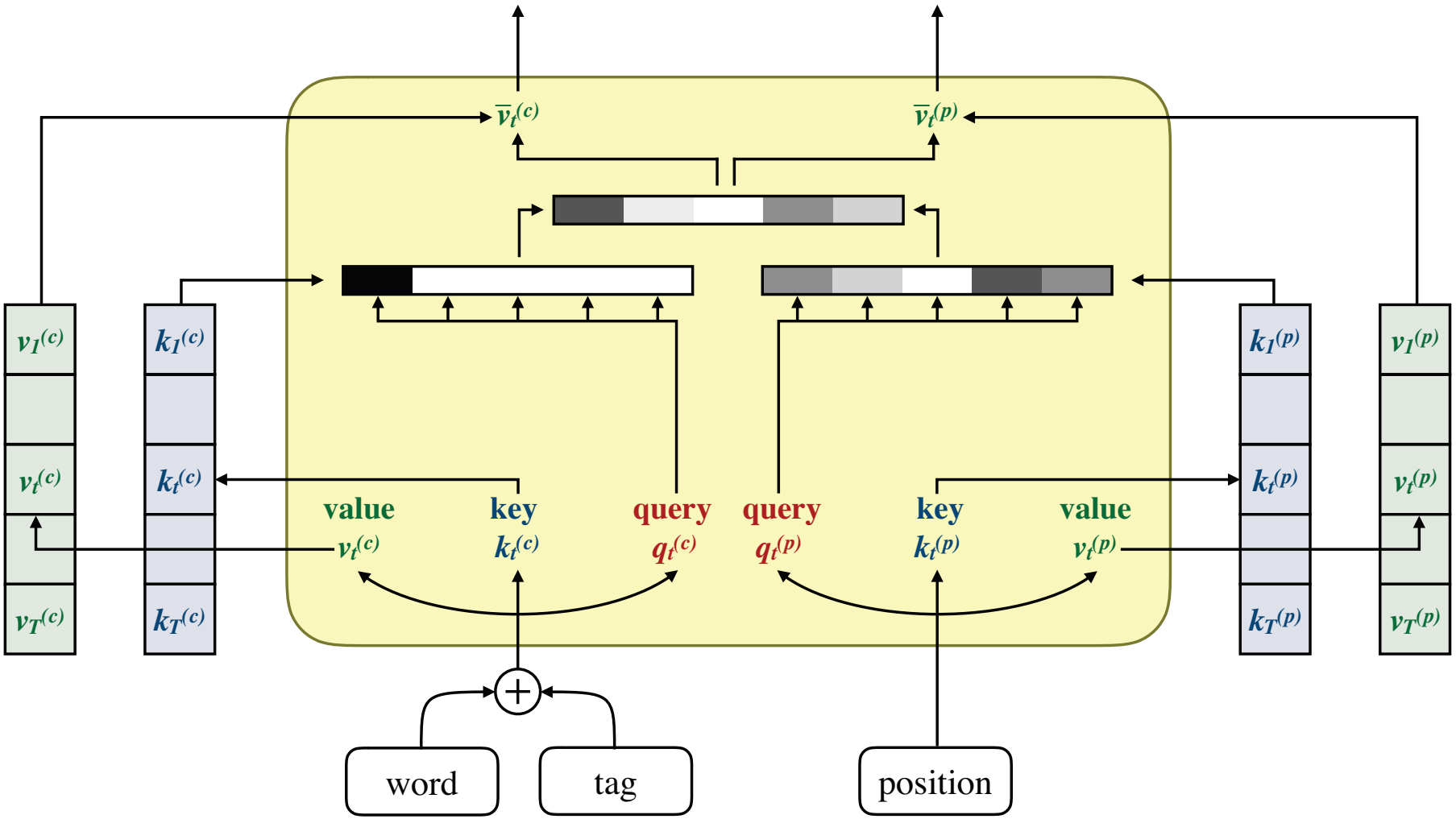


Single Attention Head



Here, $P(i \to j)$ is the probability that word i attends to word j.

Factored Attention Head



Factored attention is a commonly used design patterns for transformer models.

Span Scores

The transformer encoder give us word-based vectors (summary vectors y_k). Now we turn these into span scores:

$$s(i, j, \cdot) = M_2 \text{relu}(\text{LayerNorm}(M_1 v + c_1)) + c_2$$

where $v = \left[\vec{y}_j - \vec{y}_i; \overleftarrow{y}_{j+1} - \overleftarrow{y}_{i+1}\right]$ and M_1, M_2 are parameter matrices and c_1, c_2 are constants.

Note that y_k, the summary vector at position k, is split into two halves $\vec{y}_k$ and $\overleftarrow{y}_k$.

We then use the span scores to decode the output, i.e., to compute the best tree.

Results

Encoder Architecture	F1 (dev)	Δ
LSTM (Gaddy et al., 2018)	92.24	-0.43
Self-attentive (Section 2)	92.67	0.00
+ Factored (Section 3)	93.15	0.48
+ CharLSTM (Section 5.1)	93.61	0.94
+ ELMo (Section 5.2)	95.21	2.54

	LR	LP	F1
Single model, WSJ only			
Vinyals et al. (2015)	–	–	88.3
Cross and Huang (2016)	90.5	92.1	91.3
Gaddy et al. (2018)	91.76	92.41	92.08
Stern et al. (2017b)	92.57	92.56	92.56
Ours (CharLSTM)	**93.20**	**93.90**	**93.55**

ELMo is a pre-trained language model similar to BERT.

Summary

- Where are grammars/parsing still used?
- Two types of parsing structures: phrase-structure and dependency (focused on phrase-structure)
- Parsers these days might be grammar-less
- Two example parsers: An LSTM-based one and a transformer-based one

References

- Nikita Kitaev and Dan Klein. Constituency parsing with a self-attentive encoder. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pages 2676–2686, Melbourne, 2018.
- Oriol Vinyals, Terry Koo, Lukasz Kaiser, Slav Petrov, Ilya Sutskever, and Geoffrey Hinton. Grammar as a foreign language. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, Advances in Neural Information Processing Systems 28, Red Hook, NY, 2015. Curran Associates.